

MLOps ((S1-25_AIMLCZG523)

ASSIGNMENT - I

MLOps Heart Disease Prediction Project

Assignment Report

Group 55

Name	BITS ID
Kiranjeet Kaur Ishar	2024aa05769
Senthilkumar K	2024aa05227
Senthil Kumar K	2024aa05233
Shiv Prasad Verma	2024aa05874
Guruprasad Mishra	2024aa05858

Table of Contents

1. Introduction
2. Setup and Installation Instructions
3. EDA and Modelling Choices
4. Experiment Tracking Summary
5. Architecture Diagram
6. CI/CD and Deployment Workflow Screenshots
7. Link to Code Repository

1. Introduction

This project implements an end-to-end MLOps pipeline for heart disease prediction using a machine learning classifier. The system is designed to predict the presence of heart disease in patients based on various clinical parameters such as age, sex, chest pain type, blood pressure, cholesterol levels, and other medical indicators.

The project demonstrates comprehensive MLOps practices including:

- Data Processing & Feature Engineering: Automated data cleaning, preprocessing, and feature transformation
- Model Training & Experimentation: Training multiple ML models (Logistic Regression, Random Forest) with MLflow tracking
- Model Deployment: Containerized Flask REST API for real-time predictions
- Container Orchestration: Kubernetes deployment with service discovery and load balancing
- Monitoring & Observability: Integrated Prometheus and Grafana for metrics visualization and system monitoring
- CI/CD Ready: Docker-based deployment pipeline with health checks and logging

2. Setup and Installation Instructions

2.1 Prerequisites

- Python 3.12
- Required packages: scikit-learn, pandas, mlflow, joblib
- Docker Desktop with Kubernetes enabled (for deployment)

2.2 Installation Steps

Step 1: Clone the Repository

```
git clone <repository-url>
cd MLOps-Heart-Disease-Project
```

Step 2: Create and Activate Virtual Environment

```
python -m venv venv
# On Linux/Mac:
source venv/bin/activate
# On Windows: venv\Scripts\activate
```

Step 3: Install Dependencies

```
pip install -r requirements.txt
```

2.3 Training the Model

The model can be trained using GitHub Actions CI/CD pipeline or manually:

Using GitHub Actions

8. Navigate to the GitHub repository Actions tab
9. Select "main" workflow and click "Run workflow"
10. The workflow runs automatically on push/pull request to main branch
11. Download the trained model artifact after completion

Manual Training

```
python src/train_model.py
```

2.4 Running the Application Locally

```
python app.py
```

The application starts on <http://localhost:8080>

2.5 Kubernetes Deployment

```
# Build Docker image
```

```
docker build -t heart-disease-app:latest .  
# Deploy to Kubernetes  
kubectl apply -f k8s/deployment.yaml  
kubectl apply -f k8s/service.yaml  
# Deploy monitoring stack  
kubectl apply -f k8s/prometheus-deployment.yaml  
kubectl apply -f k8s/grafana-deployment.yaml
```

Please refer to the [README.md](#) file in the repository for more detailed instructions

3. EDA and Modelling Choices

Please refer the below files for EDA details in the repository

- 01_data_cleaning_and_eda.ipynb
- 02_model_training

3.1 Dataset Overview

The project uses the Heart Disease dataset containing clinical parameters for predicting heart disease presence. The dataset includes 13 features and a binary target variable.

Features Description

Feature	Description
age	Age of the patient
sex	Gender of the patient (1 = male, 0 = female)
cp	Chest pain type (0-3: typical angina, atypical angina, non-anginal pain, asymptomatic)
trestbps	Resting blood pressure (mm Hg on admission)
chol	Serum cholesterol measurement (mg/dl)
fbs	Fasting blood sugar > 120 mg/dl (1 = true, 0 = false)
restecg	Resting electrocardiographic results (0-2)
thalach	Maximum heart rate achieved
exang	Exercise induced angina (1 = yes, 0 = no)
oldpeak	ST depression induced by exercise relative to rest
slope	Slope of peak exercise ST segment (0-2)
ca	Number of major vessels colored by fluoroscopy (0-3)
thal	Thalassemia (1 = normal, 2 = fixed defect, 3 = reversible defect)

3.2 Exploratory Data Analysis

The EDA process included the following steps:

- Data Quality Assessment: Checking for missing values, duplicates, and data types
- Statistical Summary: Computing descriptive statistics for all numerical features
- Distribution Analysis: Visualizing feature distributions using histograms and box plots
- Correlation Analysis: Examining relationships between features using correlation matrices
- Target Variable Analysis: Understanding class distribution and balance

3.3 Modelling Choices

Two classification algorithms were selected for this project:

Logistic Regression

- Interpretable model providing probability estimates
- Works well with linearly separable data
- Fast training and prediction times
- Good baseline model for binary classification

Random Forest

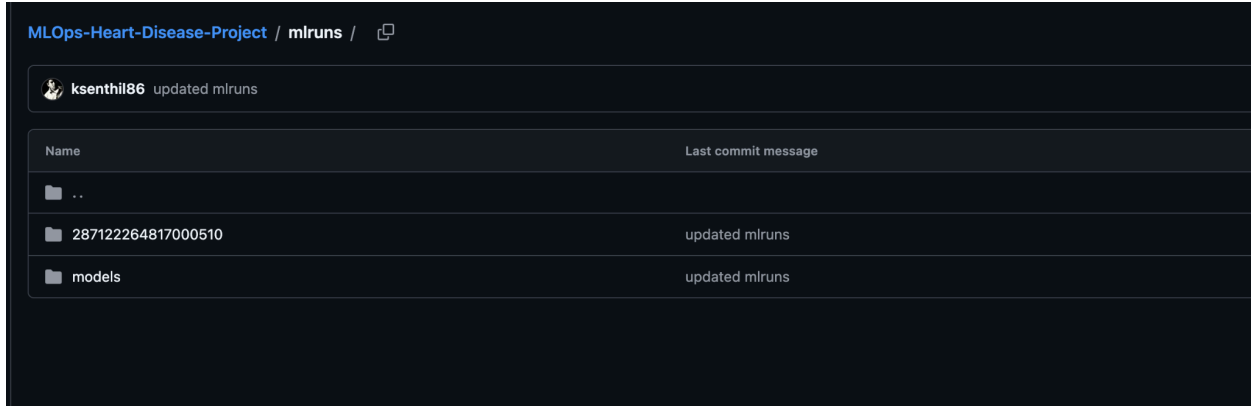
- Ensemble method providing robust predictions
- Handles non-linear relationships effectively
- Built-in feature importance metrics
- Reduces overfitting through bagging
- Selected as the final production model based on performance

3.4 Model Selection Rationale

The Random Forest classifier was selected as the final model (heartDisease_Classifier.pkl) due to its superior performance metrics and robustness. The model provides confidence scores alongside predictions, enabling risk stratification in clinical decision support.

4. Experiment Tracking Summary

Please refer the artifacts under 'mlruns' folder in code repository for training history



4.1 MLflow Integration

The project uses MLflow for comprehensive experiment tracking, model versioning, and artifact management. All experiments are stored in the mlruns directory with a SQLite backend (mlflow.db).

4.2 Tracked Metrics

The following metrics are tracked for each experiment:

Metric	Description
Accuracy	Overall classification accuracy on test set
Precision	Precision score for positive class predictions
Recall	Recall/Sensitivity for detecting heart disease
F1-Score	Harmonic mean of precision and recall
Confidence Score	Model prediction confidence (e.g., 0.85)

4.3 Tracked Parameters

Parameter	Values Tracked
Model Type	LogisticRegression, RandomForest
n_estimators	Number of trees (Random Forest)

max_depth	Maximum tree depth
random_state	Seed for reproducibility
test_size	Train/test split ratio

4.4 Artifacts Stored

- Trained model files (.pkl format)
- Feature importance plots
- Confusion matrices
- Model metadata and environment specifications

4.5 Viewing Experiments

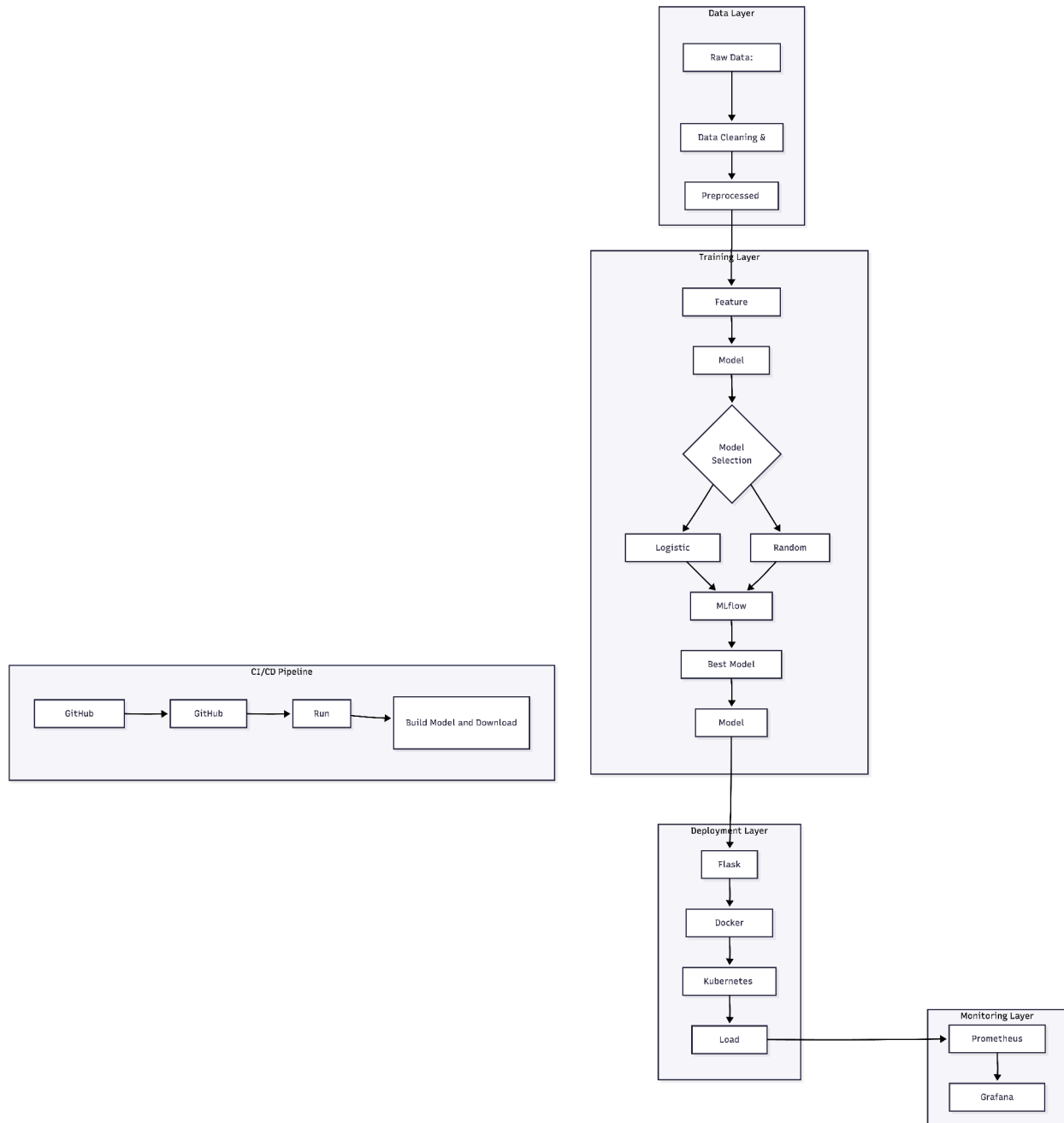
To explore training metrics and experiments locally:

```
mlflow ui
```

This launches the MLflow UI at <http://localhost:5000> where you can compare experiments, view metrics, and download artifacts.

5. Architecture Diagram

The following Mermaid diagram illustrates the complete MLOps pipeline architecture:



5.2 Component Descriptions

Component	Description
Data Layer	Raw data ingestion, cleaning, EDA, and preprocessing pipeline

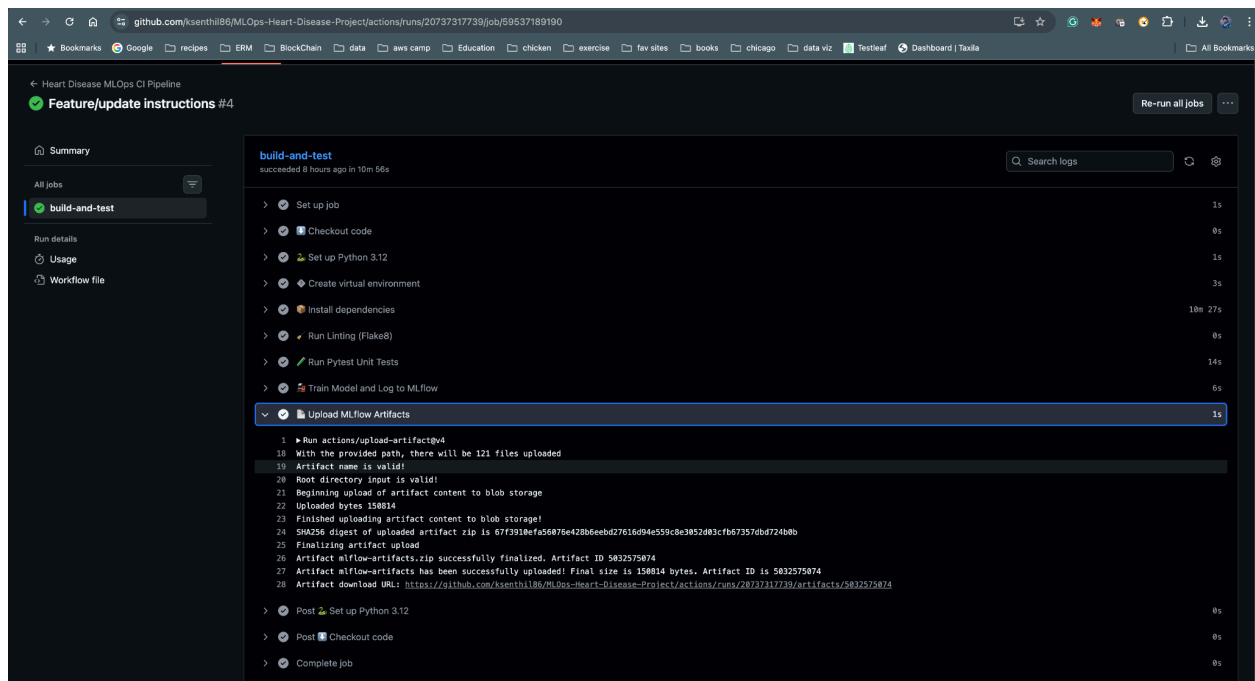
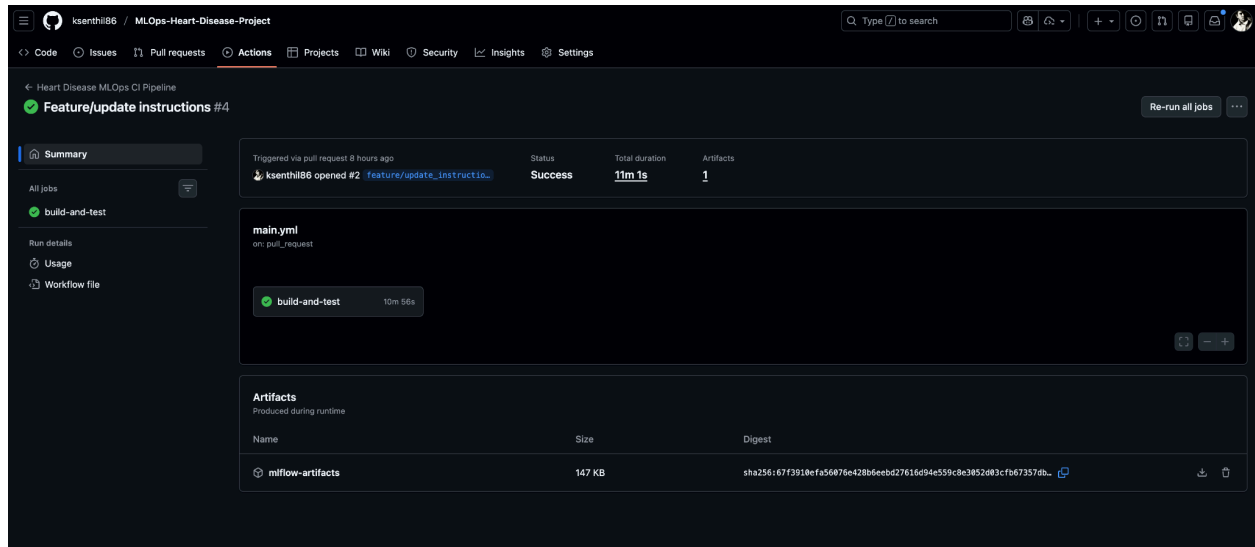
Training Layer	Feature engineering, model training with MLflow tracking, and model selection
CI/CD Pipeline	GitHub Actions workflow for automated testing and Docker image building
Deployment Layer	Flask API containerized in Docker, deployed on Kubernetes with load balancing
Monitoring Layer	Prometheus for metrics collection, Grafana for visualization and alerting

5.3 API Endpoints

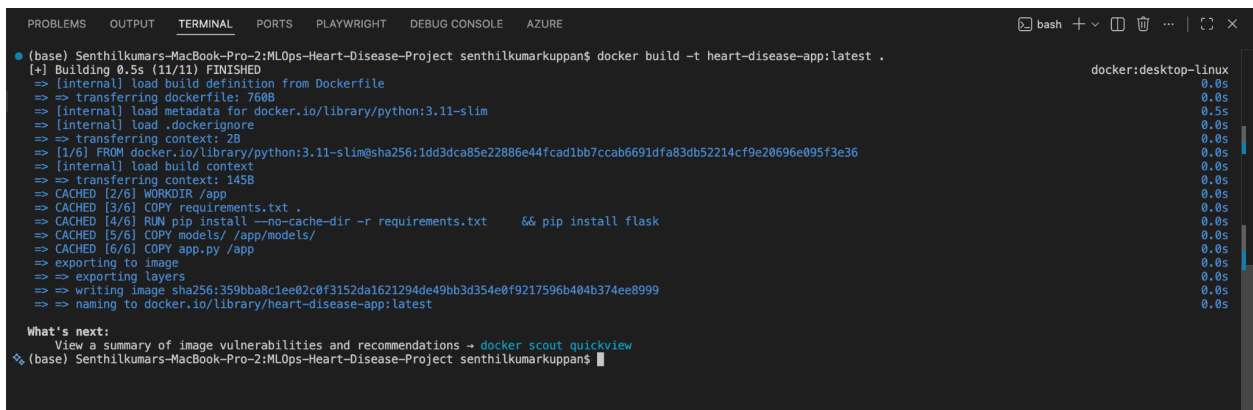
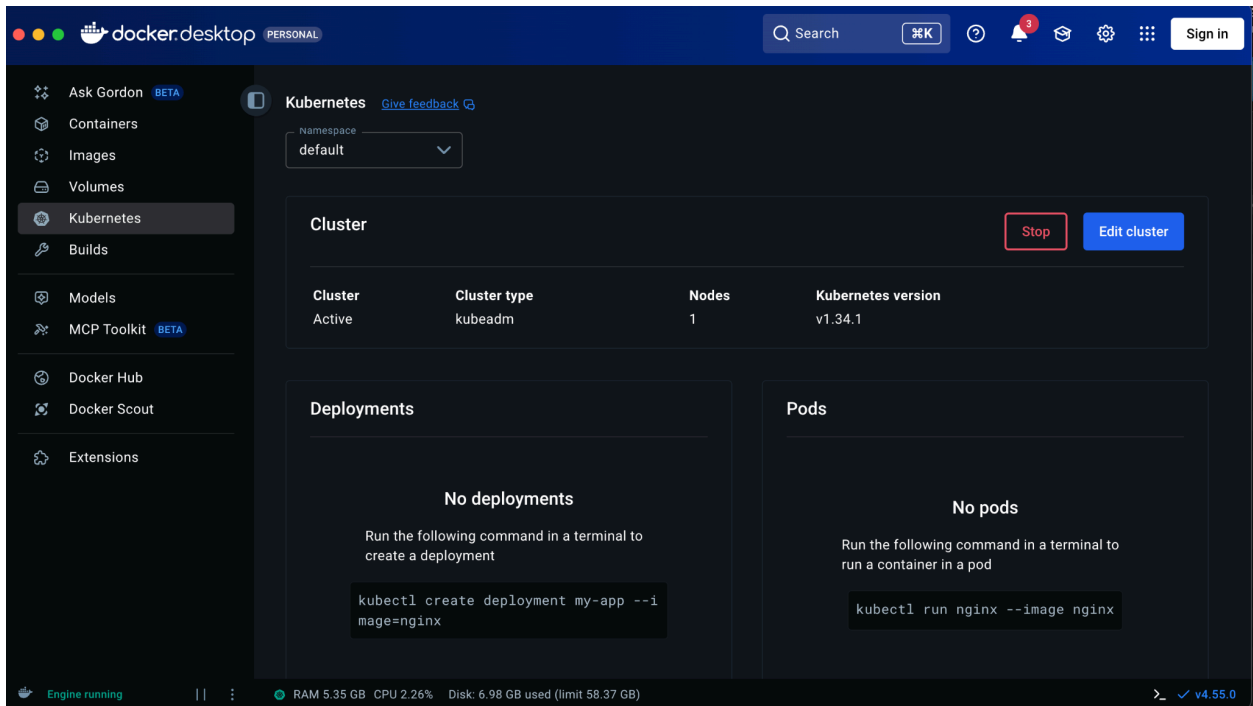
Endpoint	Method	Description
/health	GET	Health check returning model status
/predict	POST	Heart disease prediction with 13 clinical features
/metrics	GET	Prometheus metrics endpoint for monitoring

6. CI/CD and Deployment Workflow Screenshots

6.1 GitHub Actions Workflow



6.2 Docker Build Process



6.3 Kubernetes Deployment Status

```

PROBLEMS OUTPUT TERMINAL PORTS PLAYWRIGHT DEBUG CONSOLE AZURE
(bash) Senthilkumars-MacBook-Pro-2:MLOps-Heart-Disease-Project senthilkumarkuppan$ docker build -t heart-disease-app:latest .
[+] Building 0.5s (11/11) FINISHED
=> [internal] load build definition from Dockerfile
=> transferring dockerfile: 760B
=> [internal] load metadata for docker.io/library/python:3.11-slim
=> [internal] load .dockerignore
=> transferring context: 2B
=> [1/6] FROM docker.io/library/python:3.11-slim@sha256:1dd3dca85e22886e44fcad1bb7ccab6691dfa83db52214cf9e20696e095f3e36
=> [internal] load build context
=> transferring context: 145B
=> CACHED [2/6] WORKDIR /app
=> CACHED [3/6] COPY requirements.txt .
=> CACHED [4/6] RUN pip install --no-cache-dir -r requirements.txt && pip install flask
=> CACHED [5/6] COPY models/ /app/models/
=> CACHED [6/6] COPY app.py /app
=> exporting to image
=> exporting layers
=> writing image sha256:359bba8c1ee02c0f3152da1621294de49bb3d354e0f9217596b404b374ee8999
=> naming to docker.io/library/heart-disease-app:latest

What's next:
View a summary of image vulnerabilities and recommendations -> docker scout quickview
(bash) Senthilkumars-MacBook-Pro-2:MLOps-Heart-Disease-Project senthilkumarkuppan$ kubectl apply -f k8s/deployment.yaml
deployment.apps/heart-disease-app created
(bash) Senthilkumars-MacBook-Pro-2:MLOps-Heart-Disease-Project senthilkumarkuppan$ kubectl apply -f k8s/service.yaml
service/heart-disease-app-service created
(bash) Senthilkumars-MacBook-Pro-2:MLOps-Heart-Disease-Project senthilkumarkuppan$ kubectl apply -f k8s/prometheus-deployment.yaml
configmap/prometheus-config created
deployment.apps/prometheus created
service/prometheus created
serviceaccount/prometheus created
clusterrole.rbac.authorization.k8s.io/prometheus created
clusterrolebinding.rbac.authorization.k8s.io/prometheus created
(bash) Senthilkumars-MacBook-Pro-2:MLOps-Heart-Disease-Project senthilkumarkuppan$ kubectl apply -f k8s/grafana-deployment.yaml
configmap/grafana-datasources created
deployment.apps/grafana created
service/grafana created
(bash) Senthilkumars-MacBook-Pro-2:MLOps-Heart-Disease-Project senthilkumarkuppan$

```

```

(bash) Senthilkumars-MacBook-Pro-2:MLOps-Heart-Disease-Project senthilkumarkuppan$ kubectl describe deployment heart-disease-app
StrategyType:           RollingUpdate
MinReadySeconds:        0
RollingUpdateStrategy:  25% max unavailable, 25% max surge
Pod Template:
  Labels:  app=heart-disease-app
  Annotations:  prometheus.io/path: /metrics
                prometheus.io/port: 8080
                prometheus.io/scrape: true
  Containers:
    heart-disease-app:
      Image:  heart-disease-app:latest
      Port:  8080/TCP (http)
      Host Port: 0/TCP (http)
      Limits:
        cpu: 500m
        memory: 512Mi
      Requests:
        cpu: 250m
        memory: 256Mi
      Liveness:  http-get http://:8080/health delay=10s timeout=1s period=10s #success=1 #failure=3
      Readiness: http-get http://:8080/health delay=5s timeout=1s period=5s #success=1 #failure=3
      Environment:
        <none>
      Mounts:
        <none>
      Volumes:
        <none>
      Node-Selectors:
        <none>
      Tolerations:
        <none>
  Conditions:
    Type           Status  Reason
    ----           -
    Available      True    MinimumReplicasAvailable
    Progressing    True    NewReplicaSetAvailable
    OldReplicaSets: <none>
    NewReplicaSet:  heart-disease-app-65dc59f96b (2/2 replicas created)
  Events:
    Type      Reason      Age    From          Message
    ----      -
    Normal    ScalingReplicaSet   75s    deployment-controller    Scaled up replica set heart-disease-app-65dc59f96b from 0 to 2
(bash) Senthilkumars-MacBook-Pro-2:MLOps-Heart-Disease-Project senthilkumarkuppan$

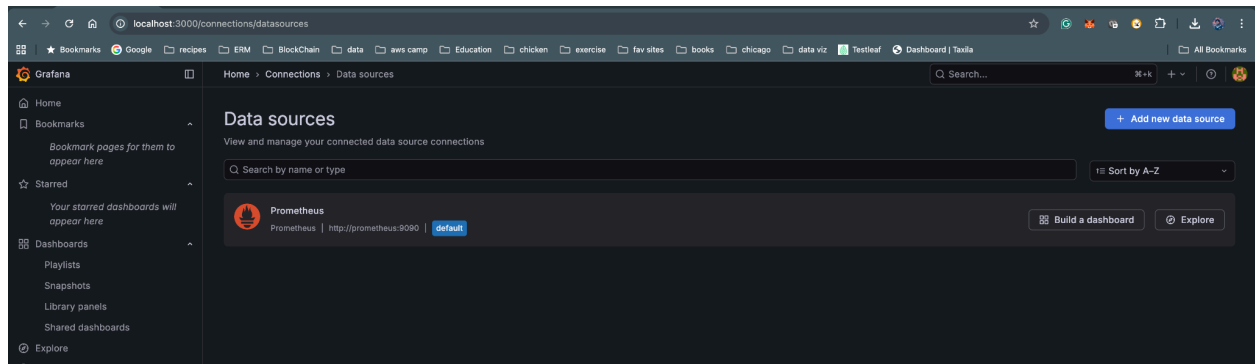
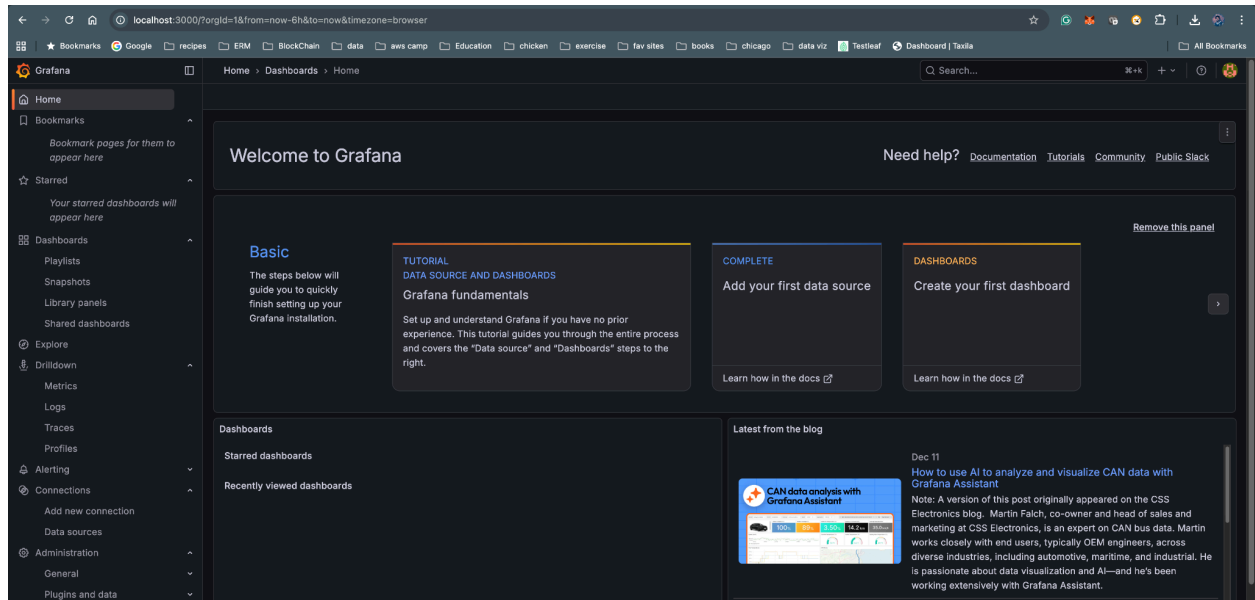
```

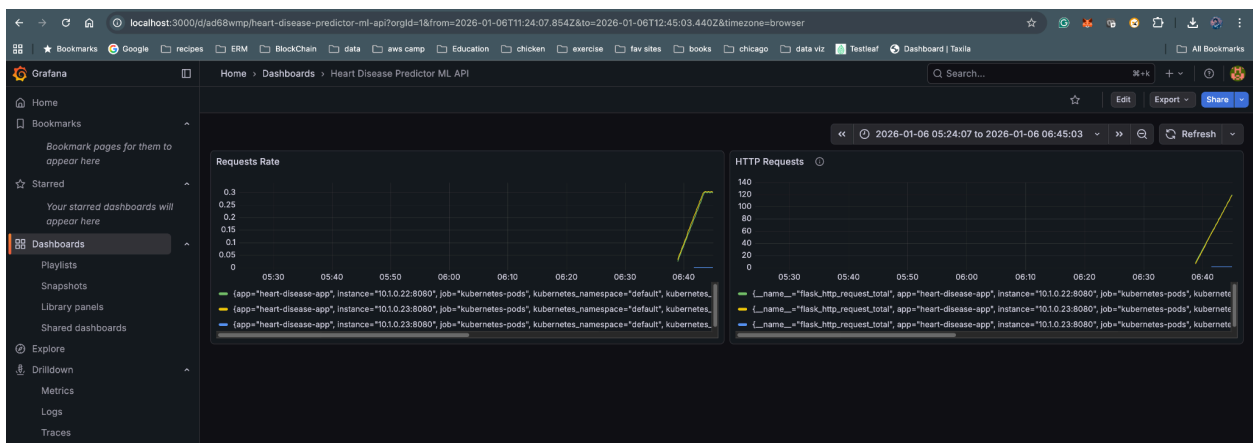
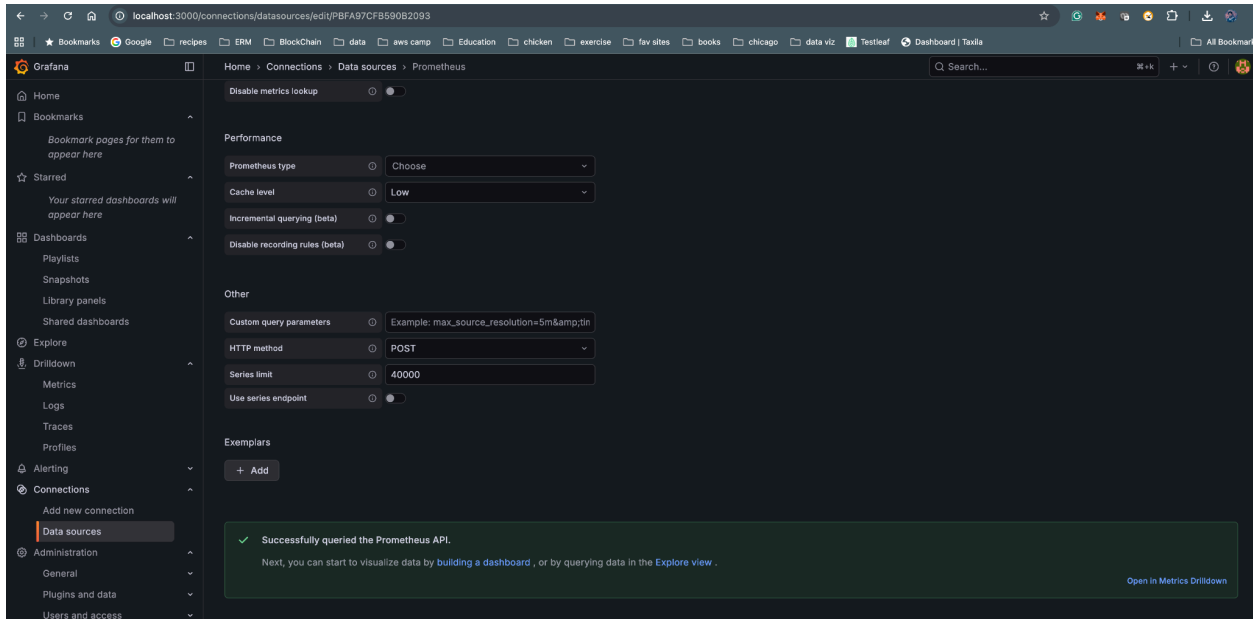
```

(bash) Senthilkumars-MacBook-Pro-2:MLOps-Heart-Disease-Project senthilkumarkuppan$ curl -X POST http://localhost/predict \
-H "Content-Type: application/json" \
-d '{
  "age": 63,
  "sex": 1,
  "cp": 3,
  "trestbps": 145,
  "chol": 233,
  "fbs": 1,
  "restecg": 0,
  "thalach": 150,
  "exang": 0,
  "oldpeak": 2.3,
  "slope": 0,
  "ca": 0,
  "thal": 1
}'
{"confidence_score":0.58,"prediction":0,"status":"Prediction successful"}
(bash) Senthilkumars-MacBook-Pro-2:MLOps-Heart-Disease-Project senthilkumarkuppan$

```

6.4 Grafana Monitoring Dashboard





6.5 Prometheus Metrics

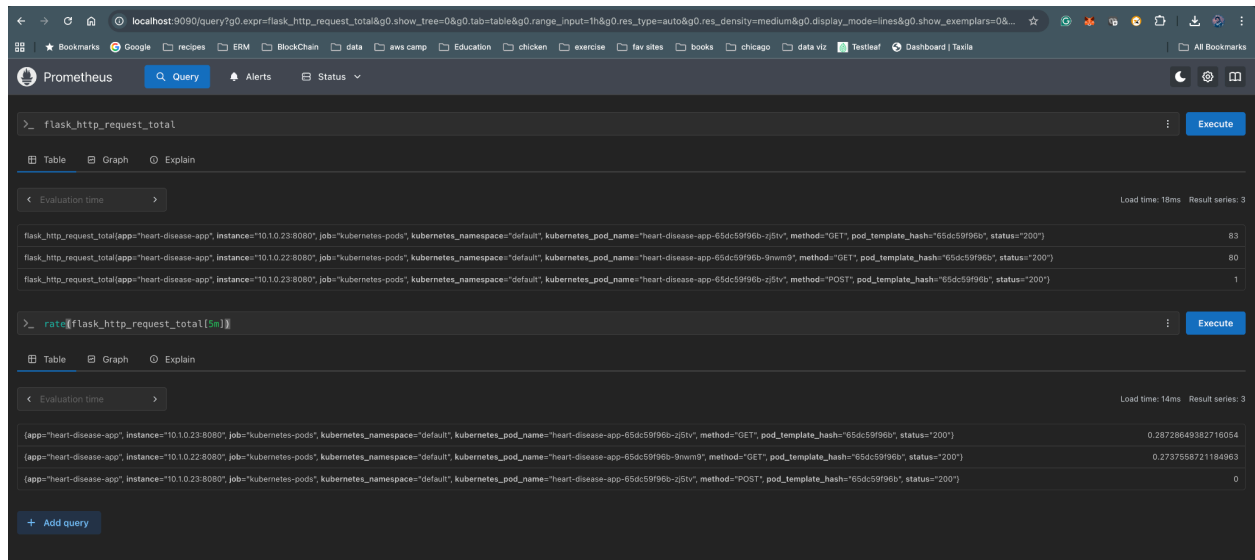
localhost:9090/targets

Prometheus Query Alerts Status > Target health

Select scrape pool Filter by target health Filter by endpoint or labels

Endpoint	Labels	Last scrape	State
http://10.1.0.22:8080/metrics	app="heart-disease-app" instance="10.1.0.22:8080" job="kubernetes-pods" kubernetes_namespace="default" kubernetes_pod_name="heart-disease-app-65dc59f9b6-rj5tv" pod_template_hash="65dc59f9b6"	4.053s ago 6ms	UP
http://10.1.0.22:8080/metrics	app="heart-disease-app" instance="10.1.0.22:8080" job="kubernetes-pods" kubernetes_namespace="default" kubernetes_pod_name="heart-disease-app-65dc59f9b6-9nm9m" pod_template_hash="65dc59f9b6"	10.829s ago 4ms	UP

2 / 2 up



7. Link to Code Repository

The complete source code for this project is available on GitHub:

<https://github.com/ksenthil86/MLOps-Heart-Disease-Project>

Repository Structure

```
MLOps-Heart-Disease-Project/  
├── .github/workflows/      # CI/CD pipeline configuration  
├── data/                  # Dataset files  
├── k8s/                   # Kubernetes manifests  
├── mlruns/                 # MLflow experiment tracking  
├── models/                # Trained model artifacts  
├── monitoring/            # Monitoring configurations  
├── src/                   # Source code modules  
├── test/                  # Unit and integration tests  
├── 01_data_cleaning_and_eda.ipynb  
├── 02_model_training.ipynb  
├── app.py                 # Flask API application  
├── Dockerfile             # Container definition  
├── requirements.txt        # Python dependencies  
└── README.md              # Project documentation
```

Demo Video

A demonstration video of the project is available at:

<https://www.youtube.com/watch?v=FD73QdivBgw>